Temco Platform

Naming · built Nov 10, 2026

Naming

Every identifier in the product follows one rule: **the namespace is the module that owns the feature.**

Patterns

Kind	Pattern	Example
Repository	`temco-<name>`	`temco-platform`, `temco-learn`
Module folder	`temco-<module>`	`extensions/temco-docs`
Extension id	`temcocontrols.temco-<module>`	`temcocontrols.temco-docs`
Command id	`temco<Module>.<action>`	`temcoDocs.validate`
Command title	`Temco <Module>: <Action>`	`Temco Docs: Validate`
Setting id	`temco<Module>.<name>`	`temcoAi.model`
View id	`temco<Module>.<view>`	`temcoDocs.pages`
Container id	`temco`, `temcoTools`, `temcoAi`	titles: Temco, Tools, Temco AI
Working area	`<module>.<name>`	`temcoRepositories.workspaceFolder`

Words

Word	Used for
Temco Platform	the product, and the application's display name
Temco Learn	the published site
Temco Docs, **Temco AI**, …	a module's name in user-visible text
`temco-docs`, `temco-ai`, …	a module in paths, ids and code

One word per thing: the product is **Temco Platform**, the published site is **Temco Learn**, and a module is named after what it does — **Temco Docs**, **Temco AI**, and so on. A module's word appears in its own

titles, its ids and its folder; nothing else may claim it.

Modules

Module	Extension id	Namespace
`temco-docs`	`temcocontrols.temco-docs`	`temcoDocs`
`temco-ai`	`temcocontrols.temco-ai`	`temcoAi`
`temco-repositories`	`temcocontrols.temco-repositories`	`temcoRepositories`
`temco-projects`	`temcocontrols.temco-projects`	`temcoProjects`
`temco-sql`	`temcocontrols.temco-sql`	`temcoSql`
`temco-nettools`	`temcocontrols.temco-nettools`	`temcoNettools`
`temco-bacnet`	`temcocontrols.temco-bacnet`	`temcoBacnet`
`temco-device-manager`	`temcocontrols.temco-device-manager`	`temcoDevices`

Timing

An identifier is free to change **before the first release**, and expensive afterwards: an extension id owns its storage folder, a setting id holds a user's value, and a command id is written into keybindings, menus and user scripts.

So the rule is: settle an identifier before it ships, and change it later only in a release that migrates what it holds. Storage is derived from the id — `<globalStorageUri>/catalog/catalog.db` for the catalog — so it moves with the id and is left behind when the id changes.